Problem Set #3

CS 6083 – Principles of Database Systems

Spring 2026

Ronit Tushir

Note: The SQL files for Question 4 (`hw3_q4.sql`) and output log (`hw3_q4_output.txt`) are submitted alongside this PDF. All queries were executed via `psql` on PostgreSQL 14. The GitHub repository for Question 1 is linked below.

1 Airline Flights and Booking – Web Application

The web application was built using Python (Flask) with PostgreSQL as the backend, using the same schema and data from HW2.

GitHub Repository: https://github.com/ronitios96/pds-flights-app

1.1 Overview

The application supports three pages:

(a) **Search Page** – A form where users enter source and destination airport codes and select a date range (start and end dates). Submitting the form triggers a search.

(b) **Results Page** – Displays all flights matching the given route and date range, showing flight number, departure date, origin code, destination code, and departure time. Flights are shown regardless of whether they are fully booked.

(c) **Flight Detail Page** – Clicking on a flight shows the aircraft capacity and the number of available seats for that specific flight number and departure date combination. Available seats are computed as capacity minus the number of bookings.

1.2 Implementation Details

- **Backend:** Flask (Python), connecting to PostgreSQL via `psycopg2`.
- **Database:** The `hw2_q2` database with the same tables (Airport, Aircraft, FlightService, Flight, Passenger, Booking) and data from HW2.
- **Key queries:**
  - Search joins `Flight` with `FlightService` on `flight_number`, filtering by origin/destination codes and date range.
  - Seat availability joins `Flight` with `Aircraft` to get capacity, and counts rows in `Booking` for the given flight and date.

2 Normal Forms

Given: $R = \{C, D, E, H, G, J\}$ with functional dependencies:

$$F = \{G \to C,\ D \to CE,\ H \to JEG,\ E \to H,\ H \to J,\ C \to E\}$$

2.1 (a) Candidate Keys

To find candidate keys, I need to figure out which attributes can determine all of $R = \{C, D, E, H, G, J\}$.

Step 1: Check which attributes never appear on the right side of any FD.

Right-hand sides across all FDs: $\{C, E, J, G, H\}$.

So D never appears on the right side of any FD. This means D must be in every candidate key (since no other attribute can derive D).

Similarly, G appears on the right only in $H \to JEG$, and D doesn't determine G directly – let me check if D alone can reach everything.

Step 2: Compute $\{D\}^+$.

- Start: $\{D\}$
- $D \to CE$: $\{D, C, E\}$
- $C \to E$: already have E
- $E \to H$: $\{D, C, E, H\}$
- $H \to JEG$: $\{D, C, E, H, J, G\}$

$\{D\}^+ = \{C, D, E, G, H, J\} = R$. So D alone determines everything.

Step 3: Check if D is minimal.

D is a single attribute, so it can't be reduced further.

Conclusion: The only candidate key is $\boxed{\{D\}}$.

(I verified that no other single attribute works: $\{G\}^+ = \{G, C, E, H, J\}$ – missing D. $\{C\}^+ = \{C, E, H, J, G\}$ – missing D. $\{E\}^+ = \{E, H, J, G, C\}$ – missing D. $\{H\}^+ = \{H, J, E, G, C\}$ – missing D. $\{J\}^+ = \{J\}$ – missing everything. So D is the unique candidate key.)

2.2 (b) Canonical Cover

Starting with $F = \{G \to C,\ D \to CE,\ H \to JEG,\ E \to H,\ H \to J,\ C \to E\}$.

Step 1: Decompose into singleton right-hand sides.

$$F_1 = \{G \to C,\ D \to C,\ D \to E,\ H \to J,\ H \to E,\ H \to G,\ E \to H,\ H \to J,\ C \to E\}$$

There's a duplicate $H \to J$, so:

$$F_1 = \{G \to C,\ D \to C,\ D \to E,\ H \to J,\ H \to E,\ H \to G,\ E \to H,\ C \to E\}$$

Step 2: Check for extraneous attributes on left sides.

All left-hand sides are single attributes, so nothing to remove here.

Step 3: Check for redundant FDs.

For each FD, I check if it can be derived from the remaining ones:

- $G \to C$: Remove it. Can we derive C from $\{G\}$ using the rest? $\{G\}^+ = \{G\}$ (nothing else has G on the LHS). No, we can't. **Keep.**
- $D \to C$: Remove it. $\{D\}^+$ using $\{G \to C, D \to E, H \to J, H \to E, H \to G, E \to H, C \to E\}$: $D \to E \to H \to G \to C$. We get C. **Redundant – remove.**
- $D \to E$: Remove it. $\{D\}^+$ using $\{G \to C, H \to J, H \to E, H \to G, E \to H, C \to E\}$: $\{D\}^+ = \{D\}$. Can't derive anything. **Keep.**
- $H \to J$: Remove it. $\{H\}^+$ using $\{G \to C, D \to E, H \to E, H \to G, E \to H, C \to E\}$: $H \to E, H \to G \to C$. We get $\{H, E, G, C\}$ but not J. **Keep.**
- $H \to E$: Remove it. $\{H\}^+$ using $\{G \to C, D \to E, H \to J, H \to G, E \to H, C \to E\}$: $H \to G \to C \to E$. We get E. **Redundant – remove.**
- $H \to G$: Remove it. $\{H\}^+$ using $\{G \to C, D \to E, H \to J, E \to H, C \to E\}$: $\{H\}^+ = \{H, J\}$. Can't reach G. **Keep.**
- $E \to H$: Remove it. $\{E\}^+$ using $\{G \to C, D \to E, H \to J, H \to G, C \to E\}$: $\{E\}^+ = \{E\}$. Can't reach H. **Keep.**
- $C \to E$: Remove it. $\{C\}^+$ using $\{G \to C, D \to E, H \to J, H \to G, E \to H\}$: $\{C\}^+ = \{C\}$. Can't reach E. **Keep.**

Step 4: Recombine.

Grouping by left-hand side:

$$F_c = \{G \to C,\ D \to E,\ H \to JG,\ E \to H,\ C \to E\}$$

This is our canonical cover: $\boxed{F_c = \{G \to C,\ D \to E,\ H \to JG,\ E \to H,\ C \to E\}}$

2.3 (c) BCNF Check

A schema is in BCNF if for every non-trivial FD $X \to Y$, X is a superkey.

The only candidate key is $\{D\}$. Let's check each FD in F_c:

- $G \to C$: G is not a superkey ($\{G\}^+ = \{G, C, E, H, J\} \neq R$). **Violates BCNF.**
- $D \to E$: D is a superkey. OK.
- $H \to JG$: H is not a superkey ($\{H\}^+$ is missing D). Violates BCNF.
- $E \to H$: E is not a superkey. Violates BCNF.
- $C \to E$: C is not a superkey. Violates BCNF.

The schema is NOT in BCNF.

BCNF Decomposition:

I'll use the standard algorithm – pick a violating FD, decompose, repeat.

Round 1: Take $G \to C$ (violates BCNF).

- $R_1 = \{G, C\}$ with FD $\{G \to C\}$. Key: G. In BCNF.

- $R' = R - \{C\} = \{D, E, G, H, J\}$. Project FDs onto R': since C is removed, $C \to E$ drops. We still have $D \to E$, $E \to H$, $H \to JG$ (all attributes still in R'). Also from $G \to C \to E$ we already have G and E in R', but the dependency was through C. Let me compute: $\{G\}^+$ in original $= \{G, C, E, H, J\}$, intersected with R' gives $\{G, E, H, J\}$, so $G \to E$ holds on R'. But $E \to H \to G$ already gives us this, so $G \to E$ is redundant given $\{E \to H, H \to G\}$.

  FDs on R': $\{D \to E,\ E \to H,\ H \to JG\}$.

Round 2: In $R' = \{D, E, G, H, J\}$ with $\{D \to E, E \to H, H \to JG\}$. Key is D ($D \to E \to H \to JG$, so $\{D\}^+ = R'$). Take $E \to H$ (violates BCNF since $\{E\}^+$ on $R' = \{E, H, J, G\}$, missing D).

- $R_2 = \{E, H, J, G\}$ with FDs $\{E \to H, H \to JG\}$. Key: E ($\{E\}^+ = \{E, H, J, G\} = R_2$). Check: $H \to JG$, is H a superkey? $\{H\}^+ = \{H, J, G\}$, missing E. Violates BCNF.
- $R'' = R' - \{H, J, G\} \cup \{E\} = \{D, E\}$ with FD $\{D \to E\}$. Key: D. In BCNF.

Round 3: In $R_2 = \{E, H, J, G\}$ with $\{E \to H, H \to JG\}$. Take $H \to JG$.

- $R_3 = \{H, J, G\}$ with FD $\{H \to JG\}$. Key: H. In BCNF.
- $R_4 = \{E, H\}$ with FD $\{E \to H\}$. Key: E. In BCNF.

Final BCNF decomposition:

$$\boxed{R_1 = \{G, C\}, \quad R_2 = \{D, E\}, \quad R_3 = \{H, J, G\}, \quad R_4 = \{E, H\}}$$

2.4 (d) Dependency Preservation

We need to check if all FDs in F_c can be verified using only attributes within a single decomposed relation.

- $G \to C$: G, C both in $R_1 = \{G, C\}$. ✓
- $D \to E$: D, E both in $R_2 = \{D, E\}$. ✓
- $H \to JG$: H, J, G all in $R_3 = \{H, J, G\}$. ✓
- $E \to H$: E, H both in $R_4 = \{E, H\}$. ✓
- $C \to E$: C is in R_1 and E is in R_2 and R_4, but no single relation contains both C and E. **Not directly preserved.**

We need to check if $C \to E$ can be inferred from the projected FDs. Using the closure test for dependency preservation: compute $\{C\}^+$ under the projected FDs.

- Start with $\{C\}$.
- In $R_1 = \{G, C\}$: $\{C\} \cap R_1 = \{C\}$, and $\{C\}^+$ restricted to R_1 gives $\{C\}$ (the only FD is $G \to C$, which doesn't fire). No change.
- No relation contains C with any FD that has C on the LHS.

$\{C\}^+$ under projected FDs = $\{C\}$, which doesn't include E.

The BCNF decomposition is NOT dependency-preserving (we lose $C \to E$).

3NF Decomposition:

Using the synthesis algorithm with $F_c = \{G \to C, D \to E, H \to JG, E \to H, C \to E\}$:

1. Create a relation for each FD in the canonical cover:
   - $S_1 = \{G, C\}$ – key G
   - $S_2 = \{D, E\}$ – key D
   - $S_3 = \{H, J, G\}$ – key H
   - $S_4 = \{E, H\}$ – key E
   - $S_5 = \{C, E\}$ – key C
2. Check if any relation contains a candidate key of R. $S_2 = \{D, E\}$ contains D, which is the candidate key. Good, no extra relation needed.
3. Remove any S_i that is a subset of another. None are subsets here.

Final 3NF decomposition:

$$\boxed{S_1 = \{G, C\},\ S_2 = \{D, E\},\ S_3 = \{H, J, G\},\ S_4 = \{E, H\},\ S_5 = \{C, E\}}$$

This preserves all dependencies (each FD is contained within one relation) and is lossless (since S_2 contains the candidate key D).

3 Normal Forms on a Real Schema

Given the schema:

`FlightSchema`(<u>flight_number</u>, airline_name, origin_code, origin_city, origin_state, origin_country, dest_code, dest_city, dest_state, dest_country, departure_time, duration, ticketPrice, plane_type, capacity)

With assumptions:

(i) origin_code and dest_code are airport codes

(ii) unique airport name, city, and state for each airport_code

(iii) country can be inferred from city and state

(iv) single price per flight stored

(v) flights between the same cities must have the same price

(vi) capacity is determined by plane_type

3.1 (a) Why the Single-Table Design is Not Optimal

This schema has several problems:

- **Redundancy:** Airport information (city, state, country) is repeated for every flight using that airport, both as origin and destination. Similarly, the capacity is stored with every flight even though it only depends on the plane type. The ticket price is repeated for all flights between the same pair of cities.
- **Update anomalies:** If Chicago's state name changed (hypothetically), we'd have to update every row where origin_city or dest_city is Chicago. If we miss some, the database becomes inconsistent. Same for capacity – if we update the capacity of a Boeing 737, we'd need to touch every flight row using that plane type.
- **Insertion anomalies:** We can't store a new airport (with its city, state, country) unless there's a flight associated with it. Similarly, we can't record a new plane type and its capacity without creating a flight.
- **Deletion anomalies:** If we delete the only flight going to some airport, we lose that airport's information entirely. Deleting the last flight using a certain plane type means we lose the capacity info for that type.

3.2 (b) Functional Dependencies

Let me use shorter names: FN = flight_number, AN = airline_name, OC = origin_code, OCI = origin_city, OS = origin_state, OCO = origin_country, DC = dest_code, DCI = dest_city, DS = dest_state, DCO = dest_country, DT = departure_time, DU = duration, TP = ticketPrice, PT = plane_type, CAP = capacity.

From the assumptions and the nature of the schema:

1. FN → AN, OC, DC, DT, DU, PT, TP
   (flight_number is the primary key, so it determines all single-valued attributes of a flight – airline, route, time, duration, plane type. Also price since assumption (iv) says single price per flight.)
2. OC → OCI, OS
   (assumption (ii): each airport code determines a unique city and state)
3. DC → DCI, DS
   (same as above but for destination)
4. OCI, OS → OCO
   (assumption (iii): country is inferred from city + state)
5. DCI, DS → DCO
   (same for destination)
6. OCI, OS, DCI, DS → TP
   (assumption (v): flights between same cities have same price. We use (city, state) pairs rather than city alone because assumption (iii) implies city names aren't globally unique – it requires both city and state to infer the country. So "same city" really means same (city, state) pair.)

7. PT → CAP
   (assumption (vi): capacity determined by plane type)

Discussion on assumptions (i) and (ii):

Assumptions (i) and (ii) are a bit tricky because the schema uses origin_code and dest_code, which are both airport codes but refer to different roles. In a normalized design, they'd both reference the same Airport table. For functional dependencies, we have to write separate FDs for origin and destination attributes (FDs 2–3 and 4–5 above), even though they follow the same logic – an airport code determines its city/state, and a city/state determines the country.

The subtlety is that we can't write a single FD like "airport_code → city, state" because the schema has two copies of these attributes (one for origin, one for destination). So we treat them independently: OC determines OCI/OS, and DC determines DCI/DS. This is essentially the same real-world constraint applied to two different columns.

3.3 (c) Candidate Keys

Starting with FN:

$\{FN\}^+$:

- FN → AN, OC, DC, DT, DU, PT, TP: {FN, AN, OC, DC, DT, DU, PT, TP}
- OC → OCI, OS: {+OCI, OS}
- DC → DCI, DS: {+DCI, DS}
- OCI, OS → OCO: {+OCO}
- DCI, DS → DCO: {+DCO}
- PT → CAP: {+CAP}

$\{FN\}^+ = \{FN, AN, OC, OCI, OS, OCO, DC, DCI, DS, DCO, DT, DU, TP, PT, CAP\} = R$.

Since FN is a single attribute and its closure is all of R, it's minimal.

No other single attribute can be a key: none of them determine FN (it doesn't appear on the right side of any FD).

The only candidate key is $\boxed{\{\textbf{flight_number}\}}$.

3.4 (d) Canonical Cover

Starting with F:

$$\begin{aligned}
&FN \to AN,\ FN \to OC,\ FN \to DC,\ FN \to DT,\ FN \to DU,\ FN \to PT,\ FN \to TP,\\
&OC \to OCI,\ OC \to OS,\\
&DC \to DCI,\ DC \to DS,\\
&OCI, OS \to OCO,\\
&DCI, DS \to DCO,\\
&OCI, OS, DCI, DS \to TP,\\
&PT \to CAP
\end{aligned}$$

Check for redundant FDs:

- FN → TP: Can we derive TP from FN without this FD? FN → OC → OCI, OS and FN → DC → DCI, DS, then OCI, OS, DCI, DS → TP. Yes! So FN → TP is **redundant – remove**.

All other FDs are essential (removing any of them would lose information since they involve attributes that can't be derived another way).

Check for extraneous attributes on left sides:
The multi-attribute left sides are {OCI, OS}, {DCI, DS}, and {OCI, OS, DCI, DS}.

- OCI, OS → OCO: Is OCI extraneous? Would OS → OCO? A state alone doesn't determine a country (e.g., "Georgia" is both a US state and a country). Keep both.
- DCI, DS → DCO: Same reasoning. Keep both.
- OCI, OS, DCI, DS → TP: Is any attribute extraneous? Removing OCI: $\{\text{OS, DCI, DS}\}^+ = \{\text{OS, DCI, DS, DCO}\}$ – can't reach TP. Removing OS: $\{\text{OCI, DCI, DS}\}^+ = \{\text{OCI, DCI, DS, DCO}\}$ – can't reach TP. By symmetry, removing DCI or DS also fails. Keep all four.

Canonical cover:

$$\boxed{F_c = \left\{ \begin{array}{l} \text{FN} \to \text{AN, OC, DC, DT, DU, PT,} \\ \text{OC} \to \text{OCI, OS,} \quad \text{DC} \to \text{DCI, DS,} \\ \text{OCI, OS} \to \text{OCO,} \quad \text{DCI, DS} \to \text{DCO,} \\ \text{OCI, OS, DCI, DS} \to \text{TP,} \quad \text{PT} \to \text{CAP} \end{array} \right\}}$$

3.5 (e) BCNF Check and Decomposition

The only candidate key is {FN}. For BCNF, every non-trivial FD must have a superkey on the left side.

- FN → ...: FN is a superkey. OK.
- OC → OCI, OS: OC is not a superkey. **Violates BCNF.**
- (Similarly DC → DCI, DS; OCI,OS → OCO; DCI,DS → DCO; OCI,OS,DCI,DS → TP; PT → CAP all violate BCNF.)

The schema is NOT in BCNF.

BCNF Decomposition:
I'll decompose step by step, each time picking a violating FD.

Round 1: OC → OCI, OS violates BCNF.

- R_1 = {OC, OCI, OS}, key: OC. But we also have OCI, OS → OCO... OCO isn't in R_1 yet. Actually, I'll handle OCO in a later step. R_1 is in BCNF.
- Remaining: remove OCI, OS from main schema (keep OC as FK reference).

Round 2: OCI, OS → OCO – but OCI and OS were moved to R_1. So we add OCO to R_1. Actually, let me reconsider and do the decomposition more carefully.

Let me approach this differently by directly giving the natural BCNF decomposition:

- R_1 = {OC, OCI, OS, OCO} with FDs {OC → OCI, OS; OCI, OS → OCO}.
  Key: OC. Check: OCI, OS → OCO – is {OCI, OS} a superkey? $\{OCI, OS\}^+ = \{OCI, OS, OCO\}$, missing OC. Violates BCNF!

So I need to further decompose R_1:

- R_{1a} = {OC, OCI, OS}, FD: OC → OCI, OS. Key: OC. BCNF. ✓
- R_{1b} = {OCI, OS, OCO}, FD: OCI, OS → OCO. Key: {OCI, OS}. BCNF. ✓

By the same logic for destination:

- R_{2a} = {DC, DCI, DS}, FD: DC → DCI, DS. Key: DC. BCNF. ✓
- R_{2b} = {DCI, DS, DCO}, FD: DCI, DS → DCO. Key: {DCI, DS}. BCNF. ✓

Next: PT → CAP:

- R_3 = {PT, CAP}, FD: PT → CAP. Key: PT. BCNF. ✓

Next: OCI, OS, DCI, DS → TP:

- R_4 = {OCI, OS, DCI, DS, TP}, FD: OCI, OS, DCI, DS → TP. Key: {OCI, OS, DCI, DS}. BCNF. ✓

Remaining attributes with FN:

- R_5 = {FN, AN, OC, DC, DT, DU, PT}, FD: FN → AN, OC, DC, DT, DU, PT. Key: FN. All determined by superkey. BCNF. ✓

Final BCNF decomposition:

R_{1a} = {OC, OCI, OS}	key: OC
R_{1b} = {OCI, OS, OCO}	key: {OCI, OS}
R_{2a} = {DC, DCI, DS}	key: DC
R_{2b} = {DCI, DS, DCO}	key: {DCI, DS}
R_3 = {PT, CAP}	key: PT
R_4 = {OCI, OS, DCI, DS, TP}	key: {OCI, OS, DCI, DS}
R_5 = {FN, AN, OC, DC, DT, DU, PT}	key: FN

Each decomposition step follows the standard BCNF algorithm (splitting on a violating FD where the common attributes form a key of one piece), which guarantees lossless-join decomposition at every step.

Note: R_{1a}/R_{1b} and R_{2a}/R_{2b} represent the same real-world entity (Airport) split into two tables. In practice, since origin and destination airports follow the same constraints, they'd reference the same Airport and CityCountry tables – but in the formal decomposition we treat them separately because the original schema had separate columns.

3.6 (f) Dependency Preservation

Check each FD in F_c:

- FN $\rightarrow$ AN, OC, DC, DT, DU, PT: all in R_5. ✓
- OC $\rightarrow$ OCI, OS: all in R_{1a}. ✓
- DC $\rightarrow$ DCI, DS: all in R_{2a}. ✓
- OCI, OS $\rightarrow$ OCO: all in R_{1b}. ✓
- DCI, DS $\rightarrow$ DCO: all in R_{2b}. ✓
- PT $\rightarrow$ CAP: all in R_3. ✓
- OCI, OS, DCI, DS $\rightarrow$ TP: all in R_4. ✓

Yes, this BCNF decomposition is dependency-preserving. All FDs from the canonical cover can be checked within a single relation. No 3NF conversion is needed.

3.7 (g) Adding Rule (vii): Same Airports $\Rightarrow$ Same Duration

The new assumption states: any flights between the same airports must have the same duration. This gives us a new FD:

$$\text{OC, DC} \rightarrow \text{DU}$$

Impact on (b) – Functional Dependencies:

We add OC, DC $\rightarrow$ DU to F. Now FN $\rightarrow$ DU becomes redundant because FN $\rightarrow$ OC, DC and OC, DC $\rightarrow$ DU.

Updated canonical cover:

$$F_c' = \left\{ \begin{array}{l} \text{FN} \rightarrow \text{AN, OC, DC, DT, PT,} \\ \text{OC} \rightarrow \text{OCI, OS,} \quad \text{DC} \rightarrow \text{DCI, DS,} \\ \text{OCI, OS} \rightarrow \text{OCO,} \quad \text{DCI, DS} \rightarrow \text{DCO,} \\ \text{OCI, OS, DCI, DS} \rightarrow \text{TP,} \quad \text{PT} \rightarrow \text{CAP,} \\ \text{OC, DC} \rightarrow \text{DU} \end{array} \right\}$$

Impact on (c) – Candidate Keys:

No change – FN is still the only candidate key. The new FD doesn't help any other attribute set reach FN.

Impact on (d) – Canonical Cover:

As shown above, we drop DU from FN $\rightarrow \ldots$ and add OC, DC $\rightarrow$ DU.

Impact on (e) – BCNF Decomposition:

OC, DC $\rightarrow$ DU violates BCNF (OC, DC is not a superkey). We need a new relation and DU gets removed from R_5:

- $R_6 = \{\text{OC, DC, DU}\}$, key: $\{\text{OC, DC}\}$. BCNF. ✓
- $R_5' = \{\text{FN, AN, OC, DC, DT, PT}\}$, key: FN. BCNF. ✓

Everything else stays the same. We now have 8 relations: $R_{1a}, R_{1b}, R_{2a}, R_{2b}, R_3, R_4, R_5', R_6$.

Impact on (f) – Dependency Preservation:

OC, DC $\rightarrow$ DU is fully contained in R_6, so the decomposition remains dependency-preserving. Still no need for 3NF.

4 Metadata Queries

All queries use the PostgreSQL `information_schema` and system catalogs, operating on the airline database from Problem 1 (same schema used in HW2).

4.1 (a) Tables with ≥ 2 Outgoing Foreign Keys to Distinct Tables

```sql
SELECT tc.table_name
FROM information_schema.table_constraints tc
JOIN information_schema.referential_constraints rc
  ON tc.constraint_name = rc.constraint_name
JOIN information_schema.table_constraints tc2
  ON rc.unique_constraint_name = tc2.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
  AND tc.table_schema = 'public'
GROUP BY tc.table_name
HAVING COUNT(DISTINCT tc2.table_name) >= 2;
```

This joins the constraint metadata to find, for each table, how many *distinct* referenced tables its foreign keys point to, then keeps only those with at least 2.

4.2 (b) Columns with Timestamp/Date Types

```sql
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
  AND data_type IN ('date', 'timestamp without time zone',
                    'timestamp with time zone',
                    'time without time zone',
                    'time with time zone',
                    'interval')
ORDER BY table_name, ordinal_position;
```

4.3 (c) Distinct Values per Attribute in the Flight Table

```sql
SELECT 'flight_number' AS attribute,
       COUNT(DISTINCT flight_number) AS distinct_count
FROM Flight
UNION ALL
SELECT 'departure_date', COUNT(DISTINCT departure_date)
FROM Flight
UNION ALL
SELECT 'plane_type', COUNT(DISTINCT plane_type)
FROM Flight;
```

Since the Flight table only has three columns, I list them explicitly. Each row counts the distinct values for one attribute.

4.4 (d) Tables with Composite Primary Keys

```sql
SELECT tc.table_name
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON tc.constraint_name = kcu.constraint_name
WHERE tc.constraint_type = 'PRIMARY KEY'
  AND tc.table_schema = 'public'
GROUP BY tc.table_name
HAVING COUNT(kcu.column_name) > 1;
```

This counts how many columns participate in each primary key constraint and keeps those with more than one.

4.5 (e) Attributes Containing the Substring "name"

```sql
SELECT table_name, column_name
FROM information_schema.columns
WHERE table_schema = 'public'
  AND column_name LIKE '%name%'
ORDER BY table_name, column_name;
```